

BigORace

Name: [Aung Myat Oo Gyaw]

Student: [6726066]

Section: [541]

Concept Abstract

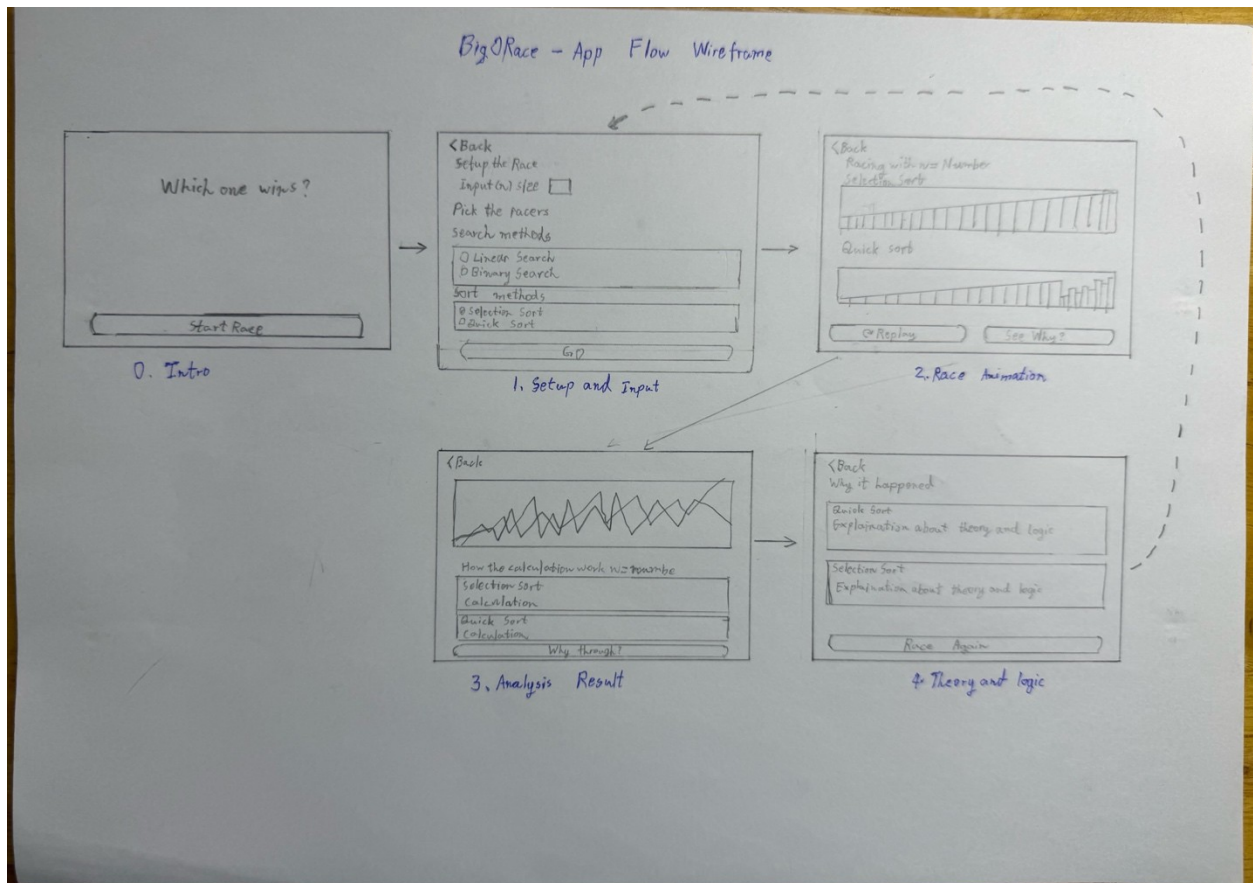
BigORace started from a pretty specific memory: sitting in a data structures class, staring at “ $O(\log n)$ ” on a slide, and nodding along without actually feeling why binary search beats linear search. The idea behind the app is simple — stop explaining Big-O and let people watch a visual comparison. You pick the input size, choose search or sort algorithms, and start the race. Each algorithm really runs on the same generated input and reports its measured operation count. The fixed-size bars illustrate each algorithm’s behavior rather than replaying every operation literally; their animation duration is paced by the measured count using logarithmic compression so slower methods remain watchable.

Textbooks tend to present time complexity as a formula you memorize, not something you experience. That disconnect is the whole problem BigORace is trying to fix. When you actually watch binary search chop its search space in half over and over while linear search plods through item by item, or watch bubble sort grind through a huge pile of comparisons while merge sort barely breaks a sweat, the difference between $O(\log n)$ and $O(n^2)$ stops being abstract. We can see what it’s happening.

On the technical side, everything is written in SwiftUI, and I was careful to keep the actual computation separate from how it is drawn on screen. Each algorithm — array lookup; linear, binary, jump, interpolation, and exponential search; plus selection, insertion, merge, quick, heap, Shell, and bubble sort — is a real Swift implementation. Every selected algorithm in a race receives the same freshly generated data, and all search methods use the same sorted array and target. The measurements run away from the main UI thread and return true elementary-operation counts. An ObservableObject view model uses a 60 fps timer to present an illustrative, log-paced race, and afterward a Swift Charts view overlays each measured result on its theoretical growth curve. The last screen ranks the results and explains, in plain language, why each algorithm finished where it did.

I think this is a fit for WWDC because it doesn’t try to do too much. It’s one idea, done carefully pick, race, reveal, reflect. There’s real engineering under the hood, but the point of the app is that moment where something clicks that used to just be a formula on a slide.

BigORace App Wireframe



Opening screen

0. Intro Front Panel

By clicking start race button it will take us to Setup and Input Panel.

1. Setup and Input Panel

In Setup and Input panel we can input number manually and there are many methods that can be measure by selecting which methods that we want to run and hit Go button this will take us to Race Animation Panel.

2. Race Animation Panel

In here we will clear see the Animation how they are working and we can replay it repeatedly then by clicking See why button this will take us to Analysis Result Panel.

3. Analysis Panel

As we can see in here Line Graph showing the performance how it takes time to complete the process and Analysis under the graph, we can also lookup the calculation too hitting Why through button this will take us to next panel call Theory and Logic Panel.

4. Theory and Logic Panel

In this Theory and Logic Panel in here explanation of theory or logic how it work to understand clearly by comparing what are the differences by solving in their own methods of that we selected and lastly when we click Race again button this will take us back to Setup and Input Panel and test the other method again.

By using this learning tool, students will understand clearly how Data Structure and Algorithm work clearly and easily. Teachers can also explain easily to students how it works by showing visual animation effects.